

CO5-AT2: EMPLOYEE PAYROLL SYSTEM (JDBC)

PROGRAM TERMINAL OUTPUT

```
Employee Payroll JDBC Output
--- Query 1: Fetch by Employee ID (101) ---
[JDBC Query] SELECT * FROM employees WHERE employee_id = 101
PermanentEmployee [ID=101 | Alice Brown | Dept=Engineering | Tax=HIGH]
Calculated Net Salary: Rs. 72000.00

--- Query 2: Fetch by Department (Engineering) ---
[JDBC Query] SELECT * FROM employees WHERE department = 'Engineering';
PermanentEmployee [ID=101 | Alice Brown | Dept=Engineering | Tax=HIGH]
Calculated Net Salary: Rs. 72000.00
PermanentEmployee [ID=103 | Carol White | Dept=Engineering | Tax=STANDARD]
Calculated Net Salary: Rs. 78750.00

--- Query 3: Catch Custom Exception Demo (ID=105) ---
[JDBC Query] SELECT * FROM employees WHERE employee_id = 105
Successfully caught exception: Negative or zero salary error: Eve Davis (Salary: -5000.0)

c:\harsha studies\Java>
```

SQL DATABASE TABLE (DB BROWSER FOR SQLITE)

DB Browser for SQLite - C:\harsha studies\Java\payroll.db

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Undo Open Project Save Project Attach Database Close Database

Database Structure Browse Data Edit Pragmas Execute SQL

Table: employees Filter in any column

	employee_id	name	department	type	hours_worked	tax_category	salary
	Filter	Filter	Filter	Filter	Filter	Filter	Filter
1	101	Harsha	Engineering	PERMANENT	0	HIGH	80000.0
2	102	Bob Smith	HR	CONTRACT	160	STANDARD	500.0
3	103	Carol White	Engineering	PERMANENT	0	STANDARD	75000.0
4	104	David Lee	Finance	CONTRACT	200	HIGH	600.0
5	105	Eve Davis	HR	PERMANENT	0	STANDARD	-5000.0

JAVA SOURCE CODE

```
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

class InvalidSalaryDataException extends Exception {
    public InvalidSalaryDataException(String msg) {
        super(msg);
    }
}

abstract class Employee {
    protected int employeeId;
    protected String name;
    protected String department;
    protected double baseSalary;
    protected String taxCategory;

    public Employee(int employeeId, String name, String department, double baseSalary) {
        this.employeeId = employeeId;
        this.name = name;
        this.department = department;
        this.baseSalary = baseSalary;
        this.taxCategory = "STANDARD";
    }

    public Employee(int employeeId, String name, String department, double baseSalary,
String taxCategory) {
        this(employeeId, name, department, baseSalary);
        this.taxCategory = taxCategory;
    }

    public abstract double calculateSalary() throws InvalidSalaryDataException;

    public int getEmployeeId() { return employeeId; }
    public String getName() { return name; }
    public String getDepartment() { return department; }

    @Override
    public String toString() {
        return String.format("ID=%-4d | %-15s | Dept=%-12s | Tax=%s",
            employeeId, name, department, taxCategory);
    }
}

class PermanentEmployee extends Employee {
    private static final double BONUS_RATE = 0.20;

    public PermanentEmployee(int id, String name, String dept, double salary) {
        super(id, name, dept, salary);
    }

    public PermanentEmployee(int id, String name, String dept, double salary, String
taxCategory) {
        super(id, name, dept, salary, taxCategory);
    }

    @Override
    public double calculateSalary() throws InvalidSalaryDataException {
        if (baseSalary <= 0)
            throw new InvalidSalaryDataException("Negative or zero salary error: " + name +
" (Salary: " + baseSalary + ")");
        double bonus = baseSalary * BONUS_RATE;
        double tax = taxCategory.equals("HIGH") ? baseSalary * 0.30 : baseSalary * 0.15;
        return baseSalary + bonus - tax;
    }
}
```

```

    }

    @Override
    public String toString() {
        return "PermanentEmployee [" + super.toString() + "]";
    }
}

class ContractEmployee extends Employee {
    private int hoursWorked;
    private double hourlyRate;

    public ContractEmployee(int id, String name, String dept, double hourlyRate, int
hoursWorked) {
        super(id, name, dept, hourlyRate);
        this.hourlyRate = hourlyRate;
        this.hoursWorked = hoursWorked;
    }

    public ContractEmployee(int id, String name, String dept, double hourlyRate, int
hoursWorked, String taxCategory) {
        super(id, name, dept, hourlyRate, taxCategory);
        this.hourlyRate = hourlyRate;
        this.hoursWorked = hoursWorked;
    }

    @Override
    public double calculateSalary() throws InvalidSalaryDataException {
        if (hourlyRate <= 0 || hoursWorked <= 0)
            throw new InvalidSalaryDataException("Invalid hourly rate/hours worked for: " +
name);
        double gross = hourlyRate * hoursWorked;
        double tax = taxCategory.equals("HIGH") ? gross * 0.25 : gross * 0.10;
        return gross - tax;
    }

    @Override
    public String toString() {
        return "ContractEmployee [" + super.toString() + "]";
    }
}

class EmployeeDAO {
    private static final String DB_URL = "jdbc:sqlite:payroll.db";

    public EmployeeDAO() {
        try {
            Class.forName("org.sqlite.JDBC");
        } catch (ClassNotFoundException e) {
        }
    }

    public void initializeDatabase() {
        String dropSQL = "DROP TABLE IF EXISTS employees;";
        String createSQL = "CREATE TABLE employees (" +
            "    employee_id INTEGER PRIMARY KEY," +
            "    name TEXT," +
            "    department TEXT," +
            "    type TEXT," +
            "    hours_worked INTEGER," +
            "    tax_category TEXT," +
            "    salary REAL" +
            ");";

        String insertSQL = "INSERT INTO employees VALUES (?, ?, ?, ?, ?, ?, ?);";

```

```

try (Connection conn = DriverManager.getConnection(DB_URL);
    Statement stmt = conn.createStatement()) {

    stmt.execute(dropSQL);
    stmt.execute(createSQL);

    try (PreparedStatement pstmt = conn.prepareStatement(insertSQL)) {
        Object[][] data = {
            {101, "Harsha", "Engineering", "PERMANENT", 0, "HIGH",
80000.0},
            {102, "Bob Smith", "HR", "CONTRACT", 160, "STANDARD",
500.0},
            {103, "Carol White", "Engineering", "PERMANENT", 0, "STANDARD",
75000.0},
            {104, "David Lee", "Finance", "CONTRACT", 200, "HIGH",
600.0},
            {105, "Eve Davis", "HR", "PERMANENT", 0, "STANDARD", -
5000.0}
        };

        for (Object[] row : data) {
            pstmt.setInt(1, (int) row[0]);
            pstmt.setString(2, (String) row[1]);
            pstmt.setString(3, (String) row[2]);
            pstmt.setString(4, (String) row[3]);
            pstmt.setInt(5, (int) row[4]);
            pstmt.setString(6, (String) row[5]);
            pstmt.setDouble(7, (double) row[6]);
            pstmt.addBatch();
        }
        pstmt.executeBatch();
    }
} catch (SQLException e) {
}

}

public List<Employee> fetchEmployeeDetails(int employeeId) throws
InvalidSalaryDataException {
    System.out.println("[JDBC Query] SELECT * FROM employees WHERE employee_id = " +
employeeId);
    List<Employee> list = new ArrayList<>();
    String sql = "SELECT * FROM employees WHERE employee_id = ?;";

    try (Connection conn = DriverManager.getConnection(DB_URL);
        PreparedStatement pstmt = conn.prepareStatement(sql)) {

        pstmt.setInt(1, employeeId);
        try (ResultSet rs = pstmt.executeQuery()) {
            while (rs.next()) {
                list.add(buildEmployeeFromResultSet(rs));
            }
        }
    } catch (SQLException e) {
    }
    return list;
}

public List<Employee> fetchEmployeeDetails(String department) throws
InvalidSalaryDataException {
    System.out.println("[JDBC Query] SELECT * FROM employees WHERE department = '" +
department + "';");
    List<Employee> list = new ArrayList<>();
    String sql = "SELECT * FROM employees WHERE department = ?;";

    try (Connection conn = DriverManager.getConnection(DB_URL);
        PreparedStatement pstmt = conn.prepareStatement(sql)) {

```

```

        pstmt.setString(1, department);
        try (ResultSet rs = pstmt.executeQuery()) {
            while (rs.next()) {
                list.add(buildEmployeeFromResultSet(rs));
            }
        }
    } catch (SQLException e) {
    }
    return list;
}

private Employee buildEmployeeFromResultSet(ResultSet rs) throws SQLException,
InvalidSalaryDataException {
    int id = rs.getInt("employee_id");
    String name = rs.getString("name");
    String dept = rs.getString("department");
    String type = rs.getString("type");
    int hours = rs.getInt("hours_worked");
    String tax = rs.getString("tax_category");
    double sal = rs.getDouble("salary");

    if (sal <= 0 && type.equals("PERMANENT")) {
        throw new InvalidSalaryDataException("Negative or zero salary error: " + name +
" (Salary: " + sal + ")");
    }

    if (type.equals("PERMANENT")) {
        return new PermanentEmployee(id, name, dept, sal, tax);
    } else {
        return new ContractEmployee(id, name, dept, sal, hours, tax);
    }
}

}

public class EmployeePayrollSystem {
    public static void main(String[] args) {
        EmployeeDAO dao = new EmployeeDAO();
        dao.initializeDatabase();

        System.out.println("--- Query 1: Fetch by Employee ID (101) ---");
        try {
            List<Employee> list = dao.fetchEmployeeDetails(101);
            for (Employee emp : list) {
                System.out.println(emp);
                System.out.printf("Calculated Net Salary: Rs. %.2f%n",
emp.calculateSalary());
            }
        } catch (InvalidSalaryDataException e) {
            System.out.println(" [Validation Error] " + e.getMessage());
        }

        System.out.println("\n--- Query 2: Fetch by Department (Engineering) ---");
        try {
            List<Employee> list = dao.fetchEmployeeDetails("Engineering");
            for (Employee emp : list) {
                System.out.println(emp);
                System.out.printf("Calculated Net Salary: Rs. %.2f%n",
emp.calculateSalary());
            }
        } catch (InvalidSalaryDataException e) {
            System.out.println(" [Validation Error] " + e.getMessage());
        }

        System.out.println("\n--- Query 3: Catch Custom Exception Demo (ID=105) ---");
        try {

```

```
        dao.fetchEmployeeDetails(105);
    } catch (InvalidSalaryDataException e) {
        System.out.println("Successfully caught exception: " + e.getMessage());
    }
}
```